1 MA2006B: Advanced Programming Evaluation

1.1 Project: Elliptic Curves over $\mathbb{R}$, $\mathbb{F}_p$, and ECDH

This project asks you to build elliptic-curve arithmetic from first principles. You will first implement the group law over the real numbers, then adapt the same ideas to a finite field, and finally use the finite-field version to simulate Elliptic Curve Diffie-Hellman (ECDH).

Final deliverable
Submit one Jupyter Notebook named `ecc_project.ipynb`. The notebook must run from top to bottom on a clean Python 3.12 installation using only the Python standard library.

1.2 1. Team Requirements

This is a team project. Each group is responsible for submitting one complete notebook.

Group requirements:

- A group may have at most three members.
- Every group member must understand the full implementation, not only the part they wrote.
- The first Markdown cell of the notebook must list the full names of all group members.
- The code must be written as one coherent solution. Avoid submitting disconnected code fragments from different members.
- All group members share responsibility for correctness, notebook execution, and compliance with the library restrictions.

Recommended division of work
One student may focus first on the real-number curve, another on modular arithmetic and finite-field addition, and another on scalar multiplication, ECDH, and tests. Before submitting, the group should review the complete notebook together and run it from a fresh kernel.

1.3 2. What You Will Build

Your notebook must define:

- `EllipticCurveReal`: arithmetic on curves over $\mathbb{R}$ using `float`.
- `EllipticCurveFp`: arithmetic on curves over a prime field $\mathbb{F}_p$ using modular arithmetic.
- `simulate_ecdh`: a function that performs an ECDH key exchange with `EllipticCurveFp`.

The point at infinity must always be represented by:

```
(None, None)
```

Important constraint
Do not import external mathematical, symbolic, numerical, or cryptographic libraries. In particular, do not use `sympy`, `cryptography`, `sage`, or `numpy`. Standard-library modules such as `math` are allowed.

1.4 3. Mathematical Background

Both classes use the short Weierstrass equation

$$E : y^2 = x^3 + ax + b.$$

The curve is valid only when it is non-singular. In this project, use

$$\Delta = 4a^3 + 27b^2$$

and require $\Delta \neq 0$ over the field being used.

For two points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$, point addition is computed by first finding a slope m, then setting

$$x_3 = m^2 - x_1 - x_2, \quad y_3 = m(x_1 - x_3) - y_1.$$

Over $\mathbb{R}$, division is ordinary real division. Over $\mathbb{F}_p$, division by d means multiplication by the modular inverse of d.

1.5 4. Suggested Notebook Order

Use Markdown cells to explain each idea before the corresponding code cell. A clear notebook should follow this order:

Part	Code to write	Purpose
1	`EllipticCurveReal`	Practice the group law with ordinary real arithmetic.
2	`EllipticCurveFp`	Translate the same formulas into arithmetic modulo a prime p.
3	`simulate_ecdh`	Use scalar multiplication to compute matching shared secrets.
4	Public assertions	Show that your implementation passes the required reference tests.

1.6 5. Part 1: Real Curve Class

Implement `EllipticCurveReal` in one code cell.

1.6.1 Required Constructor

```
def __init__(self, a: float, b: float):
```

Requirements:

- Store `a` and `b` as floats.
- Compute $\Delta = 4a^3 + 27b^2$.
- If `abs(Delta) < 1e-9`, raise `ValueError("Curve is singular")`.

1.6.2 Required Point Validation

```
def is_on_curve(self, P: tuple[float, float] | tuple[None, None]) -> bool:
```

Requirements:

- Return `True` for `(None, None)`.
- Otherwise check whether $y^2 = x^3 + ax + b$.
- Because floats are approximate, use:

```
math.isclose(left_side, right_side, rel_tol=1e-9, abs_tol=1e-9)
```

1.6.3 Required Point Addition

```
def add_points(self, P: tuple, Q: tuple) -> tuple:
```

Start by validating both input points. If either point is not on the curve, raise `ValueError("Point not on curve")`.

Handle the cases in this order:

1. Identity: if `P == (None, None)`, return `Q`; if `Q == (None, None)`, return `P`.
2. Inverses: if `x_1` and `x_2` are close and `y_1` and `-y_2` are close, return `(None, None)`.
3. Doubling: if `P == Q`, use $m = \frac{3x_1^2+a}{2y_1}$.
4. Distinct points: if `P != Q`, use $m = \frac{y_2-y_1}{x_2-x_1}$.

Then compute and return `(x_3, y_3)`.

> **Student hint**
> For the real-number class, every comparison involving coordinates should use a tolerance. Direct float equality is fragile and will fail some hidden tests.

1.7 6. Part 2: Finite-Field Curve Class

Implement `EllipticCurveFp` in a separate code cell. All coordinates and coefficients are integers modulo a prime p.

1.7.1 Required Constructor

```
def __init__(self, a: int, b: int, p: int):
```

Requirements:

- Validate `p > 2`.
- Store `self.p = p`.
- Store `self.a = a % p` and `self.b = b % p`.
- Compute `Delta = (4 * a^3 + 27 * b^2) % p`.
- If `Delta == 0`, raise `ValueError("Curve is singular modulo p")`.

1.7.2 Required Modular Inverse

```
def mod_inverse(self, k: int, p: int) -> int:
```

Requirements:

- Implement the Extended Euclidean Algorithm manually.
- Do not use `pow(k, -1, p)`.
- If `k % p == 0`, raise `ZeroDivisionError("No inverse exists")`.
- Return an integer in the range `[1, p - 1]`.

1.7.3 Required Point Validation

```
def is_on_curve(self, P: tuple[int, int] | tuple[None, None]) -> bool:
```

Requirements:

- Return `True` for `(None, None)`.
- Otherwise check

```
(y * y) % p == (x * x * x + a * x + b) % p
```

1.7.4 Required Point Addition

```
def add_points(self, P: tuple, Q: tuple) -> tuple:
```

The structure is the same as in the real case, but every division must be replaced by multiplication by a modular inverse.

Required formulas:

Case	Rule
Identity	Return the other point when one input is `(None, None)`.
Inverses	If `x_1 == x_2` and `(y_1 + y_2) % p == 0`, return `(None, None)`.
Doubling	`m = ((3 * x_1^2 + a) * self.mod_inverse(2 * y_1, p)) % p`
Distinct points	`m = ((y_2 - y_1) * self.mod_inverse(x_2 - x_1, p)) % p`
New coordinates	`x_3 = (m^2 - x_1 - x_2) % p` and `y_3 = (m * (x_1 - x_3) - y_1) % p`

1.7.5 Required Scalar Multiplication

```
def scalar_multiply(self, k: int, P: tuple) -> tuple:
```

Requirements:

- Use Double-and-Add.
- The algorithm must run in $O(\log_2 k)$ time.
- If `k == 0`, return `(None, None)`.
- If `k < 0`, replace `P` by its inverse `(x, (-y) % p)` and multiply by `-k`.
- Do not implement scalar multiplication as a loop that adds `P` exactly `k` times.

> **Student hint**
> Double-and-Add is the same idea as fast exponentiation: scan the binary representation of `k`, repeatedly double the current point, and add it to the result when the current bit is `1`.

1.8 7. Part 3: ECDH Function

Implement this function outside both classes:

```
def simulate_ecdh(curve: EllipticCurveFp, G: tuple, d_A: int, d_B: int) -> tuple:
```

Protocol:

1. Alice computes $Q_A = d_A G$ using `curve.scalar_multiply(d_A, G)`.
2. Bob computes $Q_B = d_B G$ using `curve.scalar_multiply(d_B, G)`.
3. Alice computes $S_A = d_A Q_B$.
4. Bob computes $S_B = d_B Q_A$.
5. Assert `S_A == S_B`.
6. Return `S_A`.

1.9 8. Required Public Tests

Put all public tests in the final code cell. That cell must run without errors.

1.9.1 Test Case 1: Real Numbers

Curve:

$$E : y^2 = x^3 - x + 1$$

Parameters: `a = -1.0`, `b = 1.0`

```
curve_real = EllipticCurveReal(a=-1.0, b=1.0)
assert curve_real.is_on_curve((1.0, 1.0)) == True
assert curve_real.add_points((1.0, 1.0), (1.0, 1.0)) == (-1.0, 1.0)
```

1.9.2 Test Case 2: Finite Field

Curve:

$$E : y^2 = x^3 + 2x + 2 \pmod{17}$$

Parameters: `a = 2`, `b = 2`, `p = 17`

```
curve_fp = EllipticCurveFp(a=2, b=2, p=17)
assert curve_fp.is_on_curve((5, 1)) == True
assert curve_fp.add_points((5, 1), (5, 1)) == (6, 3)
assert curve_fp.add_points((5, 1), (10, 6)) == (3, 1)
assert curve_fp.scalar_multiply(10, (5, 1)) == (7, 11)
assert curve_fp.scalar_multiply(19, (5, 1)) == (None, None)
```

1.9.3 Test Case 3: ECDH

Use the same finite-field curve and base point `G = (5, 1)`. Let Alice's private key be `d_A = 3` and Bob's private key be `d_B = 7`.

```
Q_A = curve_fp.scalar_multiply(3, (5, 1))
Q_B = curve_fp.scalar_multiply(7, (5, 1))

assert Q_A == (10, 6)
assert Q_B == (0, 6)
assert simulate_ecdh(curve_fp, (5, 1), 3, 7) == (6, 3)
```

1.10 9. Hidden Tests and Hardcoding Policy

Your notebook will also be evaluated with hidden randomized tests. These tests may use different valid curves, different prime fields, off-curve points, and different private keys.

> **Hardcoding receives zero credit**
> Do not write conditional branches that return special outputs for the public test values. Your code must implement the algebraic algorithms generally.

1.11 10. Grading Rubric

Category	Full Credit	Partial Credit	No Credit
Real curve implementation (20 pts)	Correct tolerance-based validation, identity, inverses, doubling, distinct-point addition, and hidden edge cases.	Mostly correct, but fragile float equality or missing edge cases.	Does not run, uses incorrect formulas, or hard-codes results.

Modular inverse (15 pts)	Manual Extended Euclidean Algorithm in logarithmic time.	EEA has edge-case bugs, or inverse is found by slow $O(p)$ search.	Missing, non-functional, hard-coded, or uses `pow(k, -1, p)`.
Finite-field point addition (25 pts)	Correct modular reduction and correct handling of identity, inverses, doubling, and distinct points.	Mostly correct but misses some reductions or finite-field edge cases.	Copies real arithmetic directly, fails public tests, or hard-codes outputs.
Scalar multiplication (20 pts)	Correct Double-and-Add implementation with $O(\log_2 k)$ complexity, including `k = 0` and negative `k`.	Uses Double-and-Add but fails some edge cases.	Missing, broken, or implemented as repeated addition in $O(k)$ time.
ECDH implementation (10 pts)	Computes both public keys and matching shared secrets correctly.	Protocol structure is present but some argument or key computations are wrong.	Function missing or shared secrets do not match.
Notebook quality (10 pts)	One clean notebook, Python 3.12 standard library only, sequential execution, clear explanations, and useful type hints.	Runs but explanations, formatting, or type hints are incomplete.	Notebook does not run, executes out of order, or depends on prohibited libraries.

1.12 11. Submission Checklist

Before submitting, verify:

- The first Markdown cell lists all group members.
- The group has at most three members.
- The file is named `ecc_project.ipynb`.
- The notebook runs from a fresh kernel with execution counts `[1], [2], [3], ...`.
- No prohibited libraries are imported.
- Public tests are in the final code cell.
- The implementation works for general inputs, not only for the public examples.